Separating Language Orchestration from Deterministic Control in Agentic Meal Recommendation

Sai Teja Paladi, CSE 798 Independent Study

November 2026

Abstract

Large language models can translate natural-language requests into tool calls, but delegating a complete recommendation process to a model confounds language, tool-use, and recommendation errors. We study a hybrid BEACON meal workflow in which models interpret requests while deterministic components retain identity binding, Knapsack selection, nutrition calculation, validation, and release control. A hosted agent reproduces all six direct Knapsack plans but is 28 times slower. On 30 requests and five local models, an identity gate raises complete passes from 59/150 to 79/150 and removes all ten recommender executions for unidentifiable users, while valid-workflow passes stay at 49/120 with no request changing result; a no-LLM pipeline passes 30/30. When small models select meals directly, one loses quality and variety and the other violates stored dietary conditions for half of the personas. Finally, five responsibility-based tools make the release decision an explicit model action. Over two 42-request sets with three identical repetitions, two of three models release plans their own validation call rejected; a release gate reduces such releases to zero and raises correct outcomes by up to six cases without changing any model's strict trajectory score. A keyword baseline leads on the development wording but not on held-out wording, where the two stronger gated agents are comparable to it, and models' final replies sometimes contradict the recorded state. On 48 conversations that use nicknames, misspellings, descriptions, earlier turns, and mixed Telugu–English or Hindi–English, models resolve 28–29 of 30 hard requests against 27 for rules written in advance, but pick a person for 9–12 of the 12 requests that should be clarified; using a model only to propose a person for the user to confirm executes no wrong person. The results support using models only where language understanding outperforms a declared no-LLM baseline, and then as proposers, while deterministic code retains identity, constraint-sensitive computation, and release control.
Keywords: LLM agents, tool use, meal recommendation, deterministic optimization, policy enforcement, clarification

1. Introduction

Meal recommendation combines two different kinds of work. Natural-language requests may be incomplete, ambiguous, or ask the system to ignore stored restrictions, while profile retrieval, constraint enforcement, and optimization should be reproducible. An LLM can be useful for the first kind of work, but it is not automatically the appropriate component for the second. Garimella, Srivastava, and Sheth show this distinction clearly for exact arithmetic: deterministic implementations achieve perfect accuracy with far lower latency than LLM and agentic alternatives [3]. Their architectural decision framework motivates using a model only where language understanding is useful and retaining deterministic procedures for exact work.

This paper applies that principle to the BEACON meal-recommendation setting. BEACON studies the tradeoff between nutrition and convenience for configurable, long-horizon meal recommendations [11, 12]. We do not replace or modify its Knapsack recommender. Instead, we investigate whether an LLM can reliably orchestrate the workflow around it and what happens when the user's persona is missing, unknown, or ambiguous.

A preliminary comparison motivates keeping meal selection deterministic: when small models select meals directly from the same catalog, one violates stored dietary conditions and the other loses quality and variety (Section 5.3). The main study therefore holds Knapsack fixed and asks four research questions about the workflow around it:

- **RQ1:** How reliably do small tool-calling models orchestrate a fixed deterministic recommender, and do they ask for clarification when the persona is missing, unknown, or ambiguous?
- **RQ2:** Does a deterministic identity gate make the system safe on those requests without changing what the model itself can do?

- **RQ3:** With tools organized by responsibility (profile retrieval, recommendation, nutrition analysis, validation, and release), do models choose intent-appropriate workflows, and do they withhold a plan that their own validation call reports as faulty? Does an application-level release gate change what reaches the user?
- **RQ4:** Do these request sets require a language model at all? How does a declared no-LLM pipeline compare, including on held-out wording that was not used to build it?
- **RQ5:** When requests refer to a person the way users do, with nicknames, misspellings, descriptions, earlier turns, or mixed languages, does a model identify the right person more often than deterministic rules, and how often does it pick the wrong person?

Each question has a stated hypothesis:

- **H1 (RQ1):** without the identity gate, success on valid workflows differs widely across models, and no model both completes most valid workflows and clarifies every unclear request.
- **H2 (RQ2):** with the gate, unclear requests produce no recommender executions and are all clarified, while each model's valid-workflow passes are unchanged.
- **H3 (RQ3):** (a) an LLM interprets indirect language better than a fixed keyword baseline, but performance differs by model; (b) successful recommendation paths preserve the Knapsack output exactly, because the model never selects food; (c) without the release gate, some models release plans that failed validation, and with it unsafe releases are zero while the model's strict trajectory success does not improve.
- **H4 (RQ4):** model orchestration is slower than the no-LLM pipeline, so any value it adds must come from language handling rather than computation.
- **H5 (RQ5):** (a) the best model identifies more people correctly than a stronger rule resolver; (b) every model picks a wrong person more often than those rules; (c) every condition handles exact full names.

H3 and H4 were committed before the final runs and before the held-out set was written. H1 and H2 were written after a first Experiment 2 run and before its re-run, so they predict a replication. H5 and the stronger rule resolver were committed before the Experiment 5 requests were written. Experiments 1 and 3 are descriptive.

The paper makes six bounded contributions:

1. a controlled separation between language-model orchestration and deterministic meal selection in the BEACON system, with an integration check showing exact plan preservation.
2. an identity-gated hybrid architecture that resolves a persona outside the model, constrains identity-dependent tool calls, and renders verified results without a final generative step, evaluated against the same agent without the gate across five local models.
3. a responsibility-based tool design in which release is an explicit model action, so the evaluation can observe whether a model withholds a plan its own validation rejected, and a matched release-gate comparison on development and held-out request sets with repeated runs and paired statistics.
4. a two-level, trace-based scoring scheme that separates the end state the user receives from the model's own trajectory, without an LLM judge.
5. a same-persona, same-catalog comparison between deterministic Knapsack and two local LLM-only selectors, scored after generation by the same Beacon evaluators.
6. a hard-wording identity benchmark that compares the exact-name gate, a stronger rule resolver frozen before the requests were written, and three local models on nicknames, misspellings, descriptions, earlier turns, and romanized Telugu–English and Hindi–English requests, with wrong-person identification as the safety outcome.

This is not a claim of a new optimization algorithm, a new language model, or a clinically validated recommender. It is a controlled systems study of agentic orchestration around an existing deterministic recommender.

2. Related Work

2.1 Reasoning, acting, and recommendation agents

ReAct formalizes an agent pattern in which a language model interleaves reasoning, actions, and observations from an external environment [1]. Our implementation follows the observable part of this pattern: the model receives tool schemas, chooses tool names and arguments, observes structured results, and decides whether to call another tool or stop. The presence of a particular agent framework is not what makes this loop agentic.

RecMind applies an autonomous LLM agent to recommendation using planning, personalized memory, world knowledge, and tools [2]. It demonstrates that LLMs can add flexible language understanding and planning to

recommendation tasks. Our study asks a different and narrower question. The LLM does not rank meals; it must correctly invoke an existing recommender. This boundary allows an orchestration failure to be separated from a meal-selection failure.

2.2 Choosing between deterministic and agentic computation

Doing Less with More compares native functions, service architectures, direct LLM responses, and tool-using agents on 1,000 exact expressions [3]. Its central lesson is not that agents are universally ineffective, but that architecture should follow task properties. Exact known procedures should remain deterministic, while models can be reserved for linguistic or uncertain steps. Our architecture transfers this idea from arithmetic to meal-agent workflow control: Knapsack selection and identity resolution remain deterministic; request interpretation and valid tool sequencing remain model-driven.

2.3 Evaluating tool-using agents

Tool-use evaluation requires more than judging final prose. τ-bench evaluates tool-agent-user interaction using domain policies and final environment state [4]. ToolSandbox emphasizes state dependencies, canonicalization, and insufficient-information cases [5]. The Berkeley Function-Calling Leaderboard also evaluates tool selection, arguments, multi-turn execution, and irrelevant or hallucinated calls [6]. Following this line of work, our evaluator checks the executed persona, tool order, Knapsack action, and exact returned plan rather than using an LLM to rate the response text. Like τ-bench, we score the final state the user receives; we additionally score the model's trajectory separately, because an application control can repair the final state without the model having acted correctly.

The Agentic AI Test Checklist collects 110 implementation-oriented checks for terminal state, dependency ordering, tool allowlists, runtime enforcement, and related risks [15]. It is not a meal-recommendation benchmark, so we do not claim a score over the full checklist. We adapt selected checks into task-specific oracles: all postconditions must hold together, dependent tools must follow successful prerequisites, the tool set and turn count are bounded, and plan release is enforced by the application rather than only by a prompt. We screened all 110 items against this system: 22 are measured by a scored oracle, 4 are computed as exploratory measures, 15 are approximated, 15 are relevant but need new requests (mostly multi-turn and multilingual cases), and 54 do not apply to a single-agent system with no authentication, stored records, or untrusted documents. The item-by-item screen is released with the code.

2.4 Clarification and runtime policy enforcement

Clarification is a separate capability from tool invocation. Testoni and Fernández show that model uncertainty does not necessarily match human clarification behavior [7]. Zhang and Choi likewise frame deciding when to ask as an explicit stage in ambiguity resolution [8]. These findings motivate treating missing or ambiguous identity as an observable test condition rather than assuming that a system prompt will reliably produce a question.

AgentDojo shows that tool-using agents may fail tasks and security properties even without or under prompt-injection attacks [9]. CaMeL proposes placing a protective system layer around the model and enforcing capabilities through explicit control and data flows [10]. Our policy gate is substantially smaller and does not inherit CaMeL's formal guarantees. It adopts the relevant design principle: a consequential identity-binding decision should be enforced by the application rather than delegated solely to prompt compliance. The current prototype performs identity resolution, not authentication: it does not verify that the requester is entitled to access the named profile.

2.5 BEACON and meal recommendation

BEACON formulates meal planning as a combination problem involving food roles, user constraints, convenience, nutrition, and longer-term considerations [11, 12]. Its prototype includes deterministic recommendation methods and quality measures. Our study reuses its personas, local food sources, Knapsack implementation, and evaluators. We hold these components fixed so the experiment measures orchestration rather than introducing a competing meal algorithm.

2.6 MCP and LangChain

The Model Context Protocol standardizes how AI applications discover and invoke external tools and resources [13]. LangChain provides an agent harness that can implement a model-tool loop [14]. Neither changes the scientific distinction between model-controlled orchestration and deterministic selection. The present benchmark therefore uses a small native OpenAI-compatible function-calling loop, which makes every request, response, and tool trace directly auditable. An MCP or LangChain implementation would be a useful portability or framework-overhead comparison, but it is not the novelty claimed here.

3. System Design

3.1 Fixed recommendation component

For a stored persona p, the BEACON Knapsack implementation produces a one-day plan:

$$K(p, C, d) \to M,$$

where C is the fixed local catalog, d is the fixed planning date, and M contains breakfast, lunch, and dinner role-item assignments. The same persona, catalog, date, and backend version are used in all conditions.

The language model never supplies meal IDs to Knapsack and never replaces the selected plan. Consequently, if two conditions successfully execute Knapsack for the same persona, their expected role-item assignments are identical.

3.2 Agent tools

The agent receives three logical tools:

Tool	Purpose	State effect
`list_personas()`	Return stored persona identifiers and names.	Read only
`get_persona(persona_id)`	Retrieve one stored profile.	Records retrieved identity
`run_knapsack_for_persona(persona_id)`	Execute BEACON Knapsack.	Produces the meal plan

For valid requests, the required trajectory is:

```
user request
    → model calls list_personas
    → model calls get_persona for the intended persona
    → model calls run_knapsack_for_persona for the same persona
    → verified deterministic plan
```

The model genuinely chooses each tool and its arguments. A wrong tool, wrong persona, wrong order, missing call, or repeated execution is preserved in the trace and scored as a failure.

3.3 Prompt-only architecture

In the prompt-only condition, the system prompt tells the model to identify the persona, retrieve the profile, preserve stored restrictions, ask for clarification when necessary, and invoke Knapsack. These instructions are not enforced outside the model. The model can therefore choose an unrelated persona, skip a step, provide the full name where an internal identifier is required, or run Knapsack despite insufficient information.

3.4 Identity-gated architecture

The proposed architecture adds four deterministic controls:

1. **Identity resolution.** Before model execution, application code performs a case- and punctuation-insensitive match against one explicit full name or unique first name in the stored persona set.
2. **Safe clarification.** Missing, unknown, or multiple persona matches return a deterministic clarification. The model and identity-dependent tools are not called.

3. **Argument restriction.** For a resolved request, `get_persona` and `run_knapsack_for_persona` are locked to the resolved identifier. Knapsack also requires the list and matching-profile steps to have succeeded.
4. **Trusted rendering.** After successful Knapsack execution, the application returns a short confirmation from trusted state. It does not ask the model to restate nutrition facts or invent an explanation.

This gate does not choose meals and does not complete the valid tool sequence for the model. It converts incorrect model actions into explicit failures while preventing them from becoming wrong-person recommendations.

3.5 Responsibility-based tool extension

The initial three tools expose implementation steps rather than all of the responsibilities discussed for a meal agent: no user asks to list personas. We therefore keep a tool only if it is meaningful by four criteria: it has one responsibility a domain expert would name; some request type requires it; some other request type must not use it, so calling it is a decision that can be scored; and it has its own inputs, outputs, and failure behaviour. The extension uses four such tools plus an explicit release action:

Tool	Responsibility	Enforced prerequisite
`fetch_persona_profile`	Resolve one ID, full name, or unique first name and return the stored fictional profile.	Application-resolved persona must match.
`recommend_meal_plan`	Run the unchanged deterministic Beacon Knapsack method.	Matching profile was fetched.
`calculate_nutrition_scores`	Return nutrient totals, goal fit, and FSA/WHO-style proxies when requested.	A current plan exists.
`check_plan_violations`	Detect structural and stored dietary-condition violations.	A current plan exists.
`release_plan`	Show the current plan to the user.	A current plan exists; with the release gate, it must also have passed validation.

The design began from six candidate tools: a database fetch, a persona selector, a persona retriever, a recommender, a nutrition calculator, and a violation checker. The first three are merged into `fetch_persona_profile`, because choosing a person and returning that person's record form one read-only responsibility; separate tools would add a call without a new capability, plus a new failure (select one person, retrieve another). Recommendation, nutrition analysis, and validation remain separate because they answer different intents and fail independently. `release_plan` is added so the release decision is observable. Plans are held in trusted session state, so the model does not copy a large plan into the nutrition, validation, or release tools.

The request categories make every tool selective (✓ required, ✗ must not be called):

Request category	Profile	Recommend	Nutrition	Validate	Release
Profile only	✓	✗	✗	✗	✗
Recommendation, override, indirect	✓	✓	✗	✓	✓
Nutrition analysis	✓	✓	✓	✓	✓
Injected fault	✓	✓	✗	✓	✗
Unclear identity	✗	✗	✗	✗	✗

Identity for unclear requests is resolved by application code before the model runs, rather than by a model-selected tool, because Experiment 2 shows that a model that uses tools well can still run the recommender for a person it could not identify.

Release is a separate action so that the model's decision to show a plan is an observable event. The model receives every tool result, including a failed validation, and then decides whether to call `release_plan`. For a resolved request, the application binds all calls to the resolved profile and enforces dependencies in both conditions. In

the release-gated condition, it also refuses `release_plan` unless the current plan passed validation. The ungated condition leaves that decision to the model; it does not change the prompt, schemas, model, or recommender.

The FSA-style value is the mean of available per-item traffic-light proxies using sugar, salt converted from sodium, fat, and saturated fat (range 4–12; lower is better). The WHO-style value is the mean of available custom per-item range checks over seven nutrients (range 0–7; higher is better). Coverage is reported and missing values are not imputed. Neither proxy is an official label, clinical score, or individualized advice.

For example, Earl's deterministic plan contains nine selected catalog entries, but only five have the complete fields required by both proxies. The tool therefore returns FSA-style 9.8 and WHO-style 3.8 with coverage 5/9, rather than silently filling the four missing entries. Its nutrition-goal fit is 0.506: the calorie component is 1.000, while carbohydrate, protein, and fiber fits are 0.307, 0.530, and 0.187. Nutrient totals sum available catalog values and must be read together with the missing-data fields.

4. Experimental Method

4.1 Experiment 1: integration check

Experiment 1 checks that the agentic path preserves the deterministic output before any robustness study. A direct condition receives a persona identifier and invokes Knapsack for each of the six fictional BEACON personas. An agentic condition receives a natural-language request, uses hosted `openai/gpt-oss-20b` through Groq at temperature zero, and must complete the three-tool trajectory. Because Knapsack is deterministic, identical plans are expected whenever the model supplies the correct identifier, so this is a correctness check of the integration rather than a research result. We report exact role-item fidelity, tool-sequence compliance, BEACON's goodness and quality scores, post-hoc nutrition-goal fit, and latency. The tested Knapsack implementation does not optimize the numeric calorie, carbohydrate, protein, and fiber targets; nutrition fit is observed alignment.

4.2 Experiment 2: request robustness

Experiment 2 tests RQ1 and RQ2 with 30 fixed requests over the same six stored personas. Twenty-four requests require recommendation: six each use an exact full name, a paraphrase, a unique first name, or a restriction-override request that asks the system to ignore saved restrictions. Six requests require clarification because identity is unknown, missing, or ambiguous.

Five local Ollama models are evaluated:

- `llama3.2:1b`;
- `llama3.2:3b`;
- `phi4-mini:3.8b`;
- `qwen3.5:4b`;
- `granite4:3b`.

Each model receives every request in both the prompt-only and identity-gated architectures, producing 300 case-level executions. A no-LLM pipeline also applies the same name resolver and then runs Knapsack directly. Runs are sequential to avoid local memory contention. Temperature is zero, the turn limit is eight, each model turn may generate up to 1,024 tokens, and the same tool schemas, backend, reference plans, and scoring rules are used.

An earlier version of this experiment capped each model turn at 160 tokens. Qwen 3.5 reasons before answering, and that cap exhausted its reasoning budget before it produced an answer or tool call: most of its recorded failures were empty replies. We reproduced the effect directly (the same request ends with `finish_reason = length` and no output at 160 tokens, and with a correct tool call after 281 tokens at 1,024) and re-ran all five models at 1,024 tokens so every model uses the same setting. The runner now records every turn that reaches the limit. At 1,024 tokens, 32 model turns still reach it: 30 from Llama 3.2 1B, which produces long, incoherent replies rather than reasoning, and two from Qwen.

4.3 Experiment 3: deterministic versus LLM-only selection

Experiment 3 is the motivating comparison for keeping selection deterministic. It uses the same six personas and local Beacon catalog. The deterministic condition runs Knapsack. In each LLM-only condition, the model receives the stored profile and all 124 unique candidates eligible for the requested main-course, side, or beverage roles (145 role memberships) in a compact representation containing candidate ID, name, role, dietary flags, and available

calories, carbohydrates, protein, and fiber. The LLM selects the nine catalog items directly. It cannot call Knapsack or another recommendation tool, and candidates are not pre-filtered for the persona's dietary conditions.

To measure meal choice rather than malformed ID copying, Ollama structured output constrains each field to an ID belonging to the requested role and catalog. This schema performs no ranking and applies no persona-specific dietary filter. `granite4:3b` and `llama3.2:3b` each make one call per persona at temperature zero. After selection, deterministic validation checks stored dietary conditions, and the same Beacon evaluators measure goodness, overall quality, nutrition fit, repeated foods, exact Knapsack match, tokens, and latency.

4.4 Experiment 4: meaningful tools and fault containment

Experiment 4 tests RQ3 and RQ4 with the responsibility-based tools on two request sets of 42 non-sensitive cases each. Both sets contain six cases in each of seven categories: profile-only, recommendation, nutrition analysis, restriction override, indirect intent, unclear identity, and injected fault. The **development set** is the original one. The keyword rule baseline was written while developing against it, so that set favours the baseline. The **held-out set** has the same category, persona, and label for every case but new wording, and was written after the baseline was frozen; it was not used to change the baseline, prompt, or tools. Fault-case prompts in the held-out set vary in whether they mention validation, so a model cannot rely on that hint.

Unclear identities are handled before model execution by the same deterministic resolver used in Experiment 2, in both conditions. For every other case, the model must select an intent-appropriate sequence. The loop runs until the model answers without a tool call or reaches the eight-turn limit; it never reads case labels, so a model that keeps calling tools after a profile-only request is observed doing so. Each turn may generate up to 1,024 tokens.

The injected fault removes the first required role from a completed plan after Knapsack and before validation. The validator reports the missing role to the model, which must then decide not to call `release_plan`. Without the release gate, a call releases the faulty plan; with the gate, the application refuses it. The model never receives or edits the plan's contents, so the test measures whether the model acts on the validator's report, not whether it can notice the corruption itself.

Three local Ollama models are used: `granite4:3b`, `qwen3.5:4b`, and `llama3.2:3b`. Each model runs every request in both conditions three times on each request set, giving 1,512 model-condition-request executions in total. A deterministic keyword pipeline runs both sets as a no-LLM baseline. All runs use the same tool schemas, system prompt, backend, Knapsack implementation, and temperature zero.

4.5 Scoring

A valid request passes only if it:

1. retrieves the intended profile;
2. invokes Knapsack exactly once for that persona;
3. follows `list → profile → Knapsack`;
4. exactly matches the direct Knapsack role-item assignments.

An unclear request passes only if it returns a usable clarification and does not run Knapsack. We separately count unsafe Knapsack executions, policy-blocked calls, API or loop failures, failed tools, and wall-clock latency. No generated text is graded for recommendation quality by another model.

An LLM-only plan is safe and valid only if it fills every requested meal role with a catalog member and violates none of the stored diabetes, vegetarian, vegan, or gluten-free conditions. Quality metrics are computed after generation and do not influence the model's selection.

Experiment 4 scores each case at two levels:

- **Correct outcome** asks whether the user received the right end state. A profile request must return the intended profile without creating a plan. A recommendation request must release a plan that passes validation, with nutrition analysis when requested and exact Knapsack fidelity. A fault case must detect the violation and not release the plan. An unclear request must receive a clarification without tool access.
- **Strict pass** additionally requires the model's own trajectory to be correct: exact tool membership in dependency-valid order (profile before recommendation, recommendation before analysis and validation, validation before release), no failed or policy-refused call, nutrition analysis called if and only if requested, and no API or loop error.

A gate refusal therefore cannot turn a model error into a strict pass; it can only change the outcome. We also count unsafe releases (a faulty plan released), unvalidated releases (any plan released before a passed validation), gate refusals, turns that reached the token limit, and tool calls written as plain text instead of structured calls. The metric dictionary gives each denominator and a worked example.

Two exploratory measures were added after the hypotheses were fixed and are not used to decide them. **Reply–state consistency** compares the model's final reply with the recorded state and flags three contradictions: asserting that validation passed when it failed or never ran, a release claim opposite to the recorded release, and a meal plan written out in prose when Knapsack produced none. It is a keyword check, so every flag is listed with its triggering sentence for audit. **Paraphrase consistency** pairs each development case with its held-out rewording and asks whether the release decision, the set of tools attempted, and the correct outcome are the same. They correspond to checklist items T1-002, T7-009, T7-001, and T7-006.

Repetitions at temperature 0 measure run-to-run stability of local inference rather than sampling variance. We report each repetition's count and use the per-case majority across the three repetitions for interval estimates and paired tests: 95% Wilson intervals over the 42 cases of a set, and an exact McNemar test comparing the two conditions case by case. The intervals treat cases as independent and therefore understate dependence among prompts that share a persona or category. We also report how often all three repetitions produced identical tool traces, and how often the gated trace equals the ungated trace up to the gate's first refusal, which is the expected behaviour at temperature 0 if the two conditions differ only at the gate.

4.6 Experiment 5: hard wording and identity resolution

Experiment 5 tests RQ5. Experiments 2 and 4 named every person exactly, so a name matcher sufficed. Here 48 conversations refer to people the way users do, six in each of eight categories: exact-name controls; nicknames, formal names, or a title with a surname; misspellings; descriptions using stored attributes or initials; references to earlier turns (pronouns, ordinals, corrections); romanized Telugu–English and Hindi–English requests; unknown or ambiguous references; and near-miss names, such as a stored first name with a different surname. Thirty-six are resolvable (six per persona) and twelve must be clarified. A request is labeled resolvable only when exactly one stored persona is consistent with every cue in the conversation.

Identity is studied on its own because it is where Experiments 2 and 4 found the most dangerous failure, and because a correct identification leads to the deterministic Knapsack plan already evaluated. Five conditions are compared. The **exact-name rules** are the identity gate from Experiments 2 and 4. The **stronger rules** are a deterministic resolver with common nicknames, spelling tolerance for capitalized names, titles and capitalized surnames, initials, stored attributes (diabetes, vegetarian, calorie goal), corrections, ordinals, and earlier user turns; it never guesses. It and the hypotheses were committed before any Experiment 5 request was written. In the **model** conditions, Granite 4 3B, Llama 3.2 3B, and Qwen 3.5 4B see the conversation and the stored directory (ID, name, diabetes and vegetarian flags, calorie goal), and Ollama structured output restricts the answer to the six IDs or `unclear`. Runs use temperature 0, three repetitions, and thinking disabled: with thinking enabled, a trial Qwen call used the whole 1,024-token budget without answering. A **propose-and-confirm** condition is derived from these runs without new model calls: a request whose latest message contains exactly one stored full name is executed; otherwise the model's answer is shown as a question for the user to confirm, and nothing runs.

The safety outcome is a **wrong person**: any persona other than the labeled one, including any persona on a request that must be clarified. A **missed resolution** (a clarification on a resolvable request) is safe but unhelpful.

4.7 Reproducibility

Each result records the dataset, tool-schema, and system-prompt hashes; the repository and BEACON backend commits; the local model digest; temperature, turn limit, and token limit; the complete tool trace and every assistant message; application errors; and latency. All final runs of Experiments 2 and 4 were executed from one committed source revision in a separate worktree, with outputs written outside it, and the report generators refuse results from an uncommitted worktree. All profiles are fictional and all food sources are local to the compatible BEACON checkout.

5. Results

5.1 Direct versus agentic integration check

Condition	Completed	Role coverage	Exact fidelity	Required sequence	Goodness	BEACON quality	Nutrition fit	Mean latency
Direct Knapsack	6/6	100%	100%	N/A	1.000	0.860	0.549	202.4 ms
Agentic Knapsack	6/6	100%	100%	100%	1.000	0.860	0.549	5706.4 ms

The hosted agent completes the required tool sequence and reproduces the deterministic output for all six personas. This is evidence of integration and tool-use compliance for one model, not evidence that the LLM discovers a better recommendation. Meal-quality metrics are identical because the same Knapsack method selects the food. The agentic path is approximately 28 times slower when provider retry waits are included. Excluding 23.8 seconds of recorded rate-limit waits, its mean latency is 1747.2 ms, still about 8.6 times the direct baseline.

5.2 Prompt-only, identity-gated, and no-LLM robustness

Model	Architecture	Overall	Valid requests	Unclear requests	Unsafe executions	Mean latency
None	No-LLM pipeline	**30/30**	**24/24**	**6/6**	**0**	**145.8 ms**
Granite 4 3B	Prompt-only	22/30	22/24	0/6	6	8510.9 ms
Granite 4 3B	Identity-gated	**28/30**	22/24	**6/6**	**0**	**2529.4 ms**
Llama 3.2 1B	Prompt-only	0/30	0/24	0/6	0	16247.8 ms
Llama 3.2 1B	Identity-gated	**6/30**	0/24	**6/6**	0	**6836.6 ms**
Llama 3.2 3B	Prompt-only	5/30	5/24	0/6	3	4611.2 ms
Llama 3.2 3B	Identity-gated	**11/30**	5/24	**6/6**	**0**	**2596.7 ms**
Phi-4 Mini 3.8B	Prompt-only	6/30	0/24	6/6	0	2458.1 ms
Phi-4 Mini 3.8B	Identity-gated	6/30	0/24	6/6	0	**2027.1 ms**
Qwen 3.5 4B	Prompt-only	26/30	22/24	4/6	1	22630.0 ms
Qwen 3.5 4B	Identity-gated	**28/30**	22/24	**6/6**	**0**	**12857.4 ms**

Aggregating the matched runs gives the following result:

Measure	Prompt-only	Identity-gated
Complete-case passes	59/150	**79/150**
Valid workflow passes	49/120	49/120
Unclear-request passes	10/30	**30/30**
Unsafe unclear-case Knapsack executions	10	**0**
Mean latency per case	10891.6 ms	**5369.4 ms**

H2 is supported. The gate completes all 20 unclear cases that were not already passing and introduces no regression. Paired by model and valid request, 49 requests pass in both architectures, 71 fail in both, and none

changes result. The 50.7% latency reduction comes from skipping model inference for unclear identity and omitting a final model turn after verified Knapsack execution.

H1 is supported. Valid-workflow success ranges from 0/24 (Llama 3.2 1B and Phi-4 Mini) to 22/24 (Granite and Qwen), and no model combines high valid-workflow success with complete clarification. Granite and Qwen complete 22 of 24 valid workflows but clarify only 0 and 4 of the six unclear requests; Phi-4 Mini clarifies all six and completes no valid workflow.

The completion limit changed one model's result substantially. At 160 tokens, Qwen completed 16 of 24 valid workflows, and most of its failures were empty replies. At 1,024 tokens it completes 22, matching Granite, at a mean of 22.6 s per request without the gate. The other four models' valid-workflow counts are unchanged. The earlier ranking therefore partly measured a harness limit rather than the model.

The no-LLM pipeline is both faster and more accurate on this dataset. Every valid request contains an exact full name or unique first name, every unclear case is covered by the resolver's rules, and the remaining workflow is fixed. Its 30/30 result is therefore close to true by construction. It establishes an important negative result: these 30 cases do not show that an LLM is needed. Requests involving typos, nicknames, contextual references, or free-form preferences are needed to test added language-understanding value.

The 49/120 valid-workflow total is identical because the same valid requests fail in matched prompt-only and gated runs. Across the five prompt-only runs, 49 cases pass, 41 fail in backend tool calls because the model supplies an invalid persona argument, and 30 never make a successful Knapsack call. In the gated runs, the 41 backend errors become 41 explicit policy blocks; the other 30 still lack a successful call. The gate contains invalid arguments but does not repair the model's action or force the sequence.

5.3 Deterministic versus LLM-only selection

Selector	Safe plans	Hard violations	Exact Knapsack matches	Goodness	Quality, all cases	Quality, safe only	Nutrition fit	Repeated foods	Latency
Beacon Knapsack	**6/6**	**0**	6/6 reference	**1.000**	**0.860**	0.860	0.549	**0.0**	**183.5 ms**
Granite 4 3B LLM-only	**6/6**	**0**	0/6	**1.000**	0.679	0.679	0.479	2.0	4338.9 ms
Llama 3.2 3B LLM-only	3/6	4	0/6	0.981	0.453	**0.906**	**0.563**	**0.0**	4035.3 ms

Granite returns six dietary-adherent plans, but its mean Beacon quality is 0.181 below Knapsack, nutrition fit is lower, and it repeats two foods per plan on average. Llama avoids repeated foods and slightly exceeds Knapsack's mean nutrition fit, but it violates stored dietary conditions for Jessica, Becca, and Dylan. The four violations are two non-low-sugar choices for a persona with diabetes and one meat-containing choice for each of two vegetarians. Beacon assigns zero overall quality to those unsafe plans, reducing Llama's mean quality to 0.453. Its three safe plans average 0.906, above Knapsack's 0.860. This conditional result is based on only three selected cases and must be read together with Llama's 3/6 safety rate; it prevents a claim that Knapsack has the highest conditional meal quality.

Neither model exactly reproduces a Knapsack plan. This demonstrates that the models are genuinely selecting different meals; exact disagreement alone is not an error. The safety and quality metrics show why the difference matters. Both LLMs are more than 20 times slower than direct Knapsack and consume about 12,900 prompt-plus-completion tokens across six personas, whereas Knapsack uses no model tokens.

5.4 Meaningful-tool and fault-containment result

Condition	Development strict	Development outcome	Held-out strict	Held-out outcome
No-LLM rule pipeline	**38/42**	**39/42**	**33/42**	34/42
Granite 4 3B, no gate	29/42	30/42	31/42	32/42
Granite 4 3B, release gate	29/42	36/42	31/42	**36/42**
Llama 3.2 3B, no gate	28/42	29/42	18/42	19/42
Llama 3.2 3B, release gate	28/42	29/42	18/42	19/42
Qwen 3.5 4B, no gate	19/42	29/42	24/42	30/42
Qwen 3.5 4B, release gate	19/42	35/42	24/42	**36/42**

All three repetitions produced identical tool traces for every case, model, condition, and set, so each count holds in every repetition. Each gated trace equals its ungated counterpart up to the gate's first refusal in 126 of 126 runs per model and set, except Qwen on the development set (123 of 126). The two conditions therefore differ only at the gate.

H3c is supported. Without the gate, Qwen releases all six faulty plans on both sets and Granite releases all six on the development set and four on the held-out set, each after its own validation call reported the missing meal role. On the development set Granite releases every plan it validates, whatever the result. With the gate, unsafe releases are zero for every model on both sets. Correct outcomes rise by six cases for Granite and Qwen on the development set (exact McNemar $p = 0.031$ for each), and by four for Granite ($p = 0.125$) and six for Qwen ($p = 0.031$) on the held-out set. No model's strict pass changes for any case ($p = 1$). Llama never releases a faulty plan, but it also withholds 6 of the 17 development-set plans that passed validation, so its caution is partly a failure to finish the workflow.

H3b is supported. All 558 plans released outside the fault cases exactly match the direct Knapsack plan. Llama's lower Knapsack fidelity (70.8% on the development set and 41.7% on the held-out set) reflects requests for which no plan was produced, not altered plans.

H3a is not supported. On indirect-intent requests, the baseline and Granite both pass 5/6 strictly on the development set and 6/6 on the held-out set; Llama (4/6, then 0/6) and Qwen (2/6 on both) do worse. Models differ, but none interprets indirect wording better than the keyword baseline.

The held-out set changes the comparison with the rule baseline. The baseline falls from 38 to 33 strict passes and from 39 to 34 correct outcomes, mostly on profile-only requests (3/6 to 1/6) and nutrition requests (6/6 to 3/6), where its keywords no longer match. Granite and Qwen do not fall: with the release gate both reach 36/42 correct outcomes on the held-out set (95% Wilson interval 72.2–93.3%), compared with the baseline's 34/42 (66.7–90.0%). The intervals overlap substantially, so on new wording the two gated models are comparable to the baseline rather than better. Llama falls from 28 to 18 strict passes.

Category results show where each approach fails. All three models handle every profile-only request on both sets, where the baseline manages 3/6 and then 1/6. Granite completes none of the 12 nutrition-analysis requests: it calculates nutrition and then answers without validating or releasing. Much of Qwen's lower strict score comes from extra calls, such as a second `release_plan` or an unrequested nutrition analysis; on the development set it reaches the correct outcome on all six restriction-override requests but passes none strictly. The agents take 7.5 to 36.5 s per request against about 0.25 s for the baseline, 29 to 142 times slower.

Reply–state consistency (exploratory). Final replies often contradict the recorded state, and the gate does not correct them. Without the gate, Granite asserts that validation passed when it failed or never ran in six development-set and four held-out cases, and in three per set with the gate. After releasing a plan its validator had rejected, for example, it replies "The meal plan for Becca Moore has been successfully validated and released."

Qwen states the opposite of what it did in three held-out cases, for instance saying that a plan "cannot be released" after releasing it. On the held-out set, Llama writes its own meal plan in prose in 12 cases where Knapsack never ran; all six such plans for the vegetarian personas Becca and Dylan include chicken and salmon. Because scoring uses recorded state, none of these replies earns credit, but a user reading them would be misled. The released report lists every flag with its triggering sentence.

Paraphrase consistency (exploratory). Pairing each development case with its held-out rewording, the baseline is correct on six pairs only in the development wording. Gated Granite is correct on 30 pairs in both wordings and on no pair in only one. Llama is correct on 13 pairs only in the development wording and attempts the same set of tools in only 14 of 36 pairs.

5.5 Model capability versus architecture safety

The valid-workflow total is exactly 49/120 in both Experiment 2 architectures, and every model's strict pass in Experiment 4 is identical with and without the release gate. This is the most important interpretation boundary in the study: neither gate improves the model's ability to select tools. Both change what the system permits when the model is wrong or the request is incomplete.

Phi-4 Mini illustrates the distinction. It safely clarifies all six unclear requests even without the gate but completes none of the 24 valid workflows. Granite illustrates the opposite failure: it completes 22/24 valid workflows but runs Knapsack on all six unclear prompt-only requests. In Experiment 4 the same separation appears within one model: Granite has the best trajectory of the three models on the held-out set, yet releases plans its own validation rejected. General tool-use capability, safe clarification, and acting on a validation result are empirically separable.

5.6 Hard wording and identity resolution

Condition	Correct action	Hard requests resolved	Correct clarifications	Wrong person	Missed resolutions	Mean latency
Exact-name rules	21/48	7/30	8/12	5	22	< 1 ms
Stronger rules	**44/48**	27/30	**11/12**	**3**	1	< 1 ms
Granite 4 3B	36/48	28/30	2/12	12	0	480 ms
Llama 3.2 3B	34/48	28/30	0/12	14	0	417 ms
Qwen 3.5 4B	38/48	**29/30**	3/12	10	0	762 ms

Every model gave the same answer in all three repetitions for every request, and no answer was invalid or truncated. **H5c is supported:** every condition resolves the six exact full names.

H5a is not supported. The models are good at finding a plausible person: Qwen resolves 29 of the 30 hard requests, and Granite and Llama 28, against 27 for the stronger rules and 7 for the exact-name gate. The best model's lead over the stronger rules is two requests, both resolved only by the model (exact McNemar $p = 0.5$). The rules miss a misspelling typed in lowercase ("beca moor"), a description phrased "isn't vegetarian", and a request for "the other vegetarian client" after Becca had been named.

H5b is supported, and it is the main result. The models rarely decline. Granite, Llama, and Qwen clarify only 2, 0, and 3 of the 12 requests that must be clarified, and pick a person for the rest despite an explicit instruction not to guess. Asked for "Greg Thompson", "Jessica Brown", "Earl Johnson", or "Dylan Thomas", all three return the stored person with the same first name; asked for "Megan Taylor", all three return Greg Taylor; asked for "our vegetarian client", for whichever of two named people "is easier", or for "her" with no earlier referent, they choose someone. Llama assigns the unknown "Priyanka Rao" to Greg Taylor. Wrong-person outcomes total 12, 14, and 10 for the models, against 3 for the stronger rules and 5 for the exact-name gate, whose unique-first-name rule maps four near-miss names to stored people.

Propose and confirm keeps the models' strength without their risk. Executing only requests whose latest message contains exactly one stored full name, and otherwise showing the model's answer as a question for the user to confirm, executes no wrong person for any model. It executes eight requests directly and puts the right person in front of the user on 26 or 27 more, so 34 or 35 of the 36 resolvable requests reach the right person. The cost is 10 to 14 proposals the user must decline, one for each request the model should have clarified. Model inference adds 0.4 to 0.8 s per request.

6. Discussion

6.1 Answering the research questions

Each answer states whether its hypothesis is supported, using the criteria in the metric dictionary.

RQ1 / H1 — supported. Without the identity gate, small models vary from 0/24 to 22/24 on valid workflows, and none combines high valid-workflow success with safe handling of unclear identity. The two strongest models complete 22/24 valid workflows but run the recommender for unidentifiable users or fail to clarify, while the safest clarifier completes no valid workflow.

RQ2 / H2 — supported. The deterministic gate clarifies all 30 unclear model-request pairs, eliminates all ten unsafe executions, and halves mean latency, while no valid request changes result for any model. It improves the system, not the underlying models.

RQ3 / H3 — H3b and H3c supported, H3a not supported. Responsibility-based tools make each decision observable. Plans that reach the user always match Knapsack exactly. Two of three models release plans their own validation call rejected; the release gate removes every such release on both request sets without changing any model's strict trajectory score. No model interprets indirect wording better than the keyword baseline.

RQ4 / H4 — supported; no evidence that a model is needed. Every agent is slower than the no-LLM pipeline: 2.0–22.6 s against 0.15 s per request in Experiment 2, and 7.5–36.5 s against 0.25 s in Experiment 4. The pipeline is also more accurate on Experiment 2 and on the Experiment 4 development set. On held-out wording its lead disappears, and the two stronger gated agents reach comparable correct outcomes. Neither request set shows a model adding value beyond what deterministic rules achieve; Experiment 5 tests harder wording.

RQ5 / H5 — H5b and H5c supported, H5a not supported. On hard wording, models do add value over the exact-name gate, resolving 21 or 22 more of the 30 hard requests. A rule resolver written in advance recovers almost all of that value, though, trailing the best model by two requests. The models' weakness is the opposite decision: they pick a person for nearly every request that should be clarified, so a model should not be the authority on identity. Used only to propose a person for the user to confirm, the same models reach the right person on 34 or 35 of 36 resolvable requests without executing a wrong one.

Experiments 1 and 3 have no hypothesis. In Experiment 1, one hosted model completes all six required tool trajectories and preserves the Knapsack output at substantial latency cost. In Experiment 3, Granite preserves dietary constraints but has lower all-case quality, nutrition fit, and variety than Knapsack, while Llama violates hard conditions for half of the personas, although its three safe plans have higher conditional quality. The small sample does not support a general ranking of selector quality.

6.2 What is novel

The novelty is the combination of a task-specific architecture and controlled evaluation:

- it isolates natural-language orchestration from deterministic meal selection;
- it converts persona resolution into an explicit application-level control decision instead of relying only on a system prompt;
- it evaluates the same agent with and without that gate across five local models and identical requests;
- it scores executed state—identity, tool order, Knapsack action, and exact plan fidelity—while separately reporting safety and model capability;
- it compares deterministic and direct LLM selection on the same profiles, catalog, roles, and post-hoc evaluators instead of comparing systems on incompatible inputs.
- it makes release an explicit model action, so a model's decision to show a plan its own validator rejected is observed directly, and compares release-gated and ungated runs on development and held-out wording with repeated runs, paired tests, and a check that the two conditions' traces agree up to the gate's first refusal.

The work does not claim that deterministic identity resolution is itself a new idea. Its contribution is applying and measuring that principle in a reproducible BEACON meal-agent workflow, exposing a concrete tradeoff that a final-answer accuracy metric would hide: a model can be good at valid tool trajectories and simultaneously unsafe when identity is unclear.

6.3 Architecture lesson from direct LLM selection

The direct-selection comparison clarifies the architecture tradeoff. The LLM-only selectors are not merely slower alternatives that produce the same plans. They make different tradeoffs: Granite repeatedly chooses foods despite being safe, while Llama obtains a favorable numeric nutrition-fit mean partly through plans that violate stored dietary conditions. A single aggregate metric would hide both failures.

The result supports a hybrid design in which deterministic code first enforces hard constraints and produces feasible candidates. An LLM may still add value by reranking that bounded set using soft preferences or by explaining tradeoffs. This is different from allowing the model to select from the unrestricted catalog and hoping a prompt will preserve every condition.

7. Limitations and Ethical Scope

The evaluation uses six fictional personas and 30 handcrafted requests. Exact first-name matching is deliberately conservative and does not resolve typos, nicknames, pronouns, or conversational identity. The study uses one local machine at temperature zero, with one run per model-request pair in Experiment 2 and three identical runs in Experiment 4; it does not estimate variance across sampling, hardware, or model versions. The models are small local models, so results should not be generalized to frontier hosted systems.

The responsibility-based extension adds 42 handcrafted cases but uses the same six personas, one prompt per category and persona, and three small local models. Its unclear-identity outcomes come from the pre-model resolver and do not test model clarification. Its fault set contains one synthetic corruption—removing the first required meal role—so zero unsafe releases demonstrates enforcement for that fault, not complete safety. Repetitions at temperature 0 on one machine measure run-to-run stability, not sampling variance. The held-out set was written by the same authors as the development set, so it tests new wording rather than independently authored requests. The reply–state check is keyword-based; its flags were audited by hand, but it can miss contradictions phrased in other ways. As the checklist screen records, the study does not test authorisation, injected text inside tool outputs or documents, or state carried across conversation turns beyond the identity references of Experiment 5.

Experiment 5's requests and its stronger rules were written by the same authors. The rules were committed before the requests, but knowing the categories in advance may still favor them, and with 30 hard requests the two-request difference from the best model is not significant. The romanized Telugu–English and Hindi–English requests were written by the authors, not collected from users. Models ran with thinking disabled, and the propose-and-confirm condition is derived from the same answers rather than tested with users.

The gate's six unclear cases were handcrafted for the resolver's fixed rules, so its perfect clarification result is an invariant on this test set rather than evidence of generalization. Name matching is identity resolution, not authentication: anyone can type another persona's name. A stronger prompt-only baseline with few-shot examples or an explicit `ask_clarification` tool was not tested. Those baselines and language-dependent requests are required before claiming that the gate is preferable to the strongest prompting or that an LLM adds value over the no-LLM pipeline.

The LLM-only experiment uses role- and catalog-constrained JSON generation. It therefore measures meal choice after eliminating malformed role/ID output; it does not measure fully unconstrained generation. It does not filter candidates by persona, so the observed dietary violations remain model decisions. Only two models are included in this recommendation-source comparison.

Meal plans are evaluated using existing BEACON metrics and deterministic fidelity. The study does not establish that plans are clinically appropriate, that users will follow them, or that the policy gate provides complete security. Nutrition-goal fit is post-hoc and must not be described as an optimized constraint. No real personal or protected health information is used. Because Knapsack and the BEACON evaluators come from the same system, the quality measures are not an independent human judgment and may align with the deterministic method's design objectives.

The FSA-style and WHO-style outputs are application-defined per-serving proxies, not official labels. In the worked

plan, only five of nine selected entries have complete inputs, and nutrient totals can be partial when catalog fields are missing. Coverage and missing-data fields must accompany every reported value.

The aggregate 59/150 and 79/150 observations contain repeated prompts across models and should not be treated as 150 independent human samples. Wilson intervals in Experiment 4 treat the 42 cases as independent and therefore understate dependence among prompts that share a persona or category. The current result is a controlled systems comparison, not a user study or clinical trial.

8. Next Experiment: Constrained Hybrid Reranking

The next study should add a hybrid selection condition while retaining the two completed baselines. It should hold personas, catalog, candidate visibility, and evaluation metrics fixed and compare:

1. **Deterministic Knapsack:** the existing reference selector;
2. **LLM-only selector:** the completed direct-selection condition;
3. **Constrained hybrid reranker:** deterministic code produces several feasible candidate plans, the LLM reranks them for stated soft preferences, and a deterministic validator rejects any invalid output.

Run each stochastic condition repeatedly and expand the persona/request set. Primary measures should include hard-constraint violations, BEACON goodness, quality and nutrition-goal fit, preference satisfaction, exact consistency over repeated runs, explanation faithfulness, latency, and token cost. This three-way experiment would answer whether language-model judgment adds recommendation value. It must remain separate from the orchestration experiment so a ranking change is not confused with a routing failure.

9. Conclusion

An LLM can correctly orchestrate an existing deterministic meal recommender, but success on clear requests does not ensure safe behavior. Prompt-only agents fail valid workflows, fail to clarify unclear identity, or both. A small identity gate removes every recommendation for an unidentifiable user and halves latency without changing any valid result. When validation is a separate tool and release is the model's own decision, two of three small models release plans their validator rejected, and one describes them as validated. A deterministic release gate blocks every such release, but it does not make the model act correctly or report truthfully, so the evaluation must score the model's trajectory and the resulting state separately and must not trust the model's description of what it did.

The no-LLM pipeline solves the 30-case set and leads on the development wording of the 42-case set. On held-out wording its advantage disappears but does not reverse. With harder wording, models finally help: they resolve nicknames, misspellings, and references that the exact-name gate cannot. A rule resolver written in advance comes within two requests of the best model, however, and the models guess where they should ask, so no experiment here shows a model beating carefully written rules. The direct-selection comparison reinforces the division of responsibility: one LLM sacrifices quality and variety while the other violates stored dietary conditions, and an agent that bypasses Knapsack writes the same kind of unsafe plan in prose. Use the model for language-dependent steps only where it beats a declared rule baseline, and for bounded soft-preference ranking. Where a model interprets language, let it propose and let the user or deterministic code decide; keep constraint-sensitive recommendation, identity binding, and release decisions in inspectable deterministic components; and evaluate both the trajectory and the resulting state.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations*, 2023.

[2] Y. Wang, Z. Jiang, Z. Chen, F. Yang, Y. Zhou, E. Cho, X. Fan, Y. Lu, X. Huang, and Y. Yang. "RecMind: Large Language Model Powered Agent for Recommendation." *Findings of NAACL*, pp. 4351–4364, 2024. doi:10.18653/v1/2024.findings-naacl.271.

[3] R. Garimella, B. Srivastava, and A. Sheth. "Doing Less with More: A First-Principles Exploration of the Suitability of Agentic Computing over Alternative Architectural Choices." 2026.

[4] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains." 2024.

[5] J. Lu et al. "ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities." 2024.

[6] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez. "The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models." *Proceedings of the 42nd International Conference on Machine Learning*, pp. 48371–48392, 2025.

[7] A. Testoni and R. Fernández. "Asking the Right Question at the Right Time: Human and Model Uncertainty Guidance to Ask Clarification Questions." *EACL*, pp. 258–275, 2024. doi:10.18653/v1/2024.eacl-long.16.

[8] M. J. Q. Zhang and E. Choi. "Clarify When Necessary: Resolving Ambiguity Through Interaction with LMs." *Findings of NAACL*, pp. 5541–5558, 2025. doi:10.18653/v1/2025.findings-naacl.306.

[9] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents." 2024.

[10] E. Debenedetti et al. "Defeating Prompt Injections by Design." 2025.

[11] V. Nagpal, S. L. Valluru, K. Lakkaraju, and B. Srivastava. "BEACON: Balancing Convenience and Nutrition in Meals With Long-Term Group Recommendations and Reasoning on Multimodal Recipes." 2024.

[12] V. Nagpal, S. L. Valluru, K. Lakkaraju, N. Gupta, Z. Abdulrahman, A. Davison, and B. Srivastava. "A Novel Approach to Balance Convenience and Nutrition in Meals With Long-Term Group Recommendations and Reasoning on Multimodal Recipes and its Implementation in BEACON." 2024.

[13] Model Context Protocol. "Specification." Accessed October 27, 2026.

[14] LangChain. "Agents." Accessed October 27, 2026.

[15] S. Padmanabhuni. "Agentic AI Test Checklist 110." Hugging Face dataset, 110 rows, CC BY 4.0. Accessed November 3, 2026.